

Guida strategica per tipologia di esercizio

Ripasso ICC

1 Costruzione di Macchine di Turing

1. **Capisci la condizione:** cosa deve tracciare la macchina mentre scorre l'input? (un conteggio, un pattern, una parità...)
2. **Conta quanti “stati mentali”** ti servono per rappresentare tutte le situazioni possibili (es. 2 per parità, 4 per resto mod 4, 3 per un pattern a 2 caratteri)
3. **Scrivi l'idea a parole** per ciascuno stato, prima di formalizzare
4. **Traduci ogni stato in transizioni:** per ogni simbolo possibile (di solito $0, 1, \square$), decidi cosa scrivere, dove andare, in che direzione muoverti
5. **Non dimenticare** le transizioni per $\square$ (fine stringa): decidono accept/reject
6. **Verifica con un esempio concreto:** segui la macchina passo-passo su un input piccolo, controlla che il risultato sia quello atteso
7. **Complessità:** se la macchina scorre l'input una volta sola, senza tornare indietro $\Rightarrow O(n)$

2 Dimostrare che un problema è in P / FP

1. **Scrivi lo pseudocodice** (stile libero: `for`, `if`, `return`, con $\leftarrow$ per le assegnazioni)
2. **Parte 1 – conta le istruzioni:** quante volte gira ogni ciclo, quante istruzioni per giro; il totale deve risultare $O(n^k)$ per qualche k fisso
3. **Parte 2 – dimensione dei dati intermedi:** le variabili usate durante il calcolo restano polinomiali in bit? (spesso basta notare che sono “parte dell'input” o un indice limitato da n)
4. **Parte 3 – costo di ogni istruzione:** le operazioni singole (confronto, somma, lettura/scrittura) sono polinomiali?
5. **Conclusione:** (1)+(2)+(3) polinomiali $\Rightarrow$ tempo totale polinomiale $\Rightarrow \in P$ (se decisionale) o FP (se calcola una funzione)

3 Dimostrare che un problema è in NP (certificato + verificatore)

1. **Trova il certificato:** cosa dovrebbe darti qualcuno per convincerti velocemente della risposta “sì”? (una lista di indici, un cammino, un sottoinsieme...)
2. **Verifica che il certificato sia di dimensione polinomiale** rispetto all'input
3. **Scrivi il verificatore:** uno pseudocodice che, dato l'input e il certificato, controlla in tempo polinomiale se il certificato è valido
4. **Dimostra la Parte 3 dello schema P/FP** anche per il verificatore (ogni istruzione polinomiale)
5. **Conclusione:** certificato polinomiale + verificatore polinomiale $\Rightarrow \in NP$

4 Dimostrare NP-hardness / NP-completezza

1. **Scegli un problema già noto NP-hard/NP-completo** da cui partire (3SAT, HAM-CYCLE, INDSET, VERTEX COVER...)
2. **Costruisci una funzione di riduzione f** : trasforma ogni istanza del problema noto in un'istanza del tuo problema
3. **Dimostra che f è calcolabile in tempo polinomiale** (di solito banale: è pura manipolazione sintattica)
4. **Dimostra l'equivalenza**: $x \in A \iff f(x) \in B$ (spesso in 2 direzioni separate, $\Rightarrow$ e $\Leftarrow$)
5. **Conclusione**: problema noto $\leq_p$ tuo problema $\Rightarrow$ il tuo problema è NP-hard
6. Se hai già dimostrato anche NP-membership (sezione 3): NP-hard + in NP $\Rightarrow$ **NP-completo**

Attenzione alla direzione! Per dimostrare NP-hardness, la riduzione deve andare *dal problema noto verso il tuo problema* (noto $\leq_p$ tuo). Se hai una riduzione nella direzione opposta (tuo $\leq_p$ noto), questo dimostra solo che il tuo problema *non è più difficile* del problema noto – non dimostra NP-hardness!

5 Classificare una variante di un problema NP-completo noto

1. **Individua cosa cambia**: cosa è stato rimosso, fissato, o semplificato rispetto all'originale?
2. **Chiediti se la restrizione elimina la parte “difficile”** del problema originale (di solito, una scelta combinatoria complessa)
3. **Se sì**: cerca una strategia greedy/diretta che risolva la variante in tempo polinomiale $\Rightarrow$ probabilmente è in P/FP (usa la Sezione 2)
4. **Se no**: probabilmente resta NP-completo $\Rightarrow$ dimostra sia NP-membership (Sezione 3) sia NP-hardness (Sezione 4)
5. **Non fidarti di riduzioni “date” senza controllarne la direzione**: una riduzione dalla variante verso l'originale non basta per NP-hardness (vedi il box di attenzione sopra)

6 VC-dimension

1. **Lower bound $VC \geq k$** : scegli k punti in una configurazione “comoda” (in fila su una retta, su un cerchio, a diamante...); mostra che *ogni* etichettatura possibile è realizzabile
2. **Upper bound $VC < k + 1$** : scegli $k + 1$ punti; trova **un'etichettatura specifica** che non si può realizzare. Due strategie tipiche:
 - *Punto in mezzo*: un punto “intrappolato” dentro la figura formata dagli altri (usa la convessità)
 - *Alternanza*: etichettatura D,F,D,F,... che richiede troppi “blocchi” separati
3. **Conclusione**: $VC \geq k$ e $VC < k + 1 \Rightarrow VC = k$

7 Decidibilità / Teorema di Rice

1. **Controlla se la proprietà è semantica:** dipende solo dal comportamento di M (linguaggio accettato), o dalla struttura del suo codice?
 - Se **non** è semantica: Rice non si applica. Dimostra decidibilità/indecidibilità direttamente (spesso decidibile, con un controllo sintattico banale)
2. **Controlla se è non-triviale:** trova una macchina che soddisfa la proprietà, e una che non la soddisfa
3. **Se semantica e non-triviale:** per il Teorema di Rice, è **indecidibile**

8 PAC-learning

8.1 Esercizi di calcolo numerico

1. Identifica la formula corretta per la classe di ipotesi in questione
2. Sostituisci ε, δ (e altri parametri, es. n) nella formula
3. Calcola passo-passo: prima le frazioni, poi il logaritmo (**naturale**, $\ln$), infine il prodotto

8.2 Dimostrazioni concettuali (efficiente PAC-learnability)

1. **Se vuoi dimostrare che è learnable:** identifica cosa rende “sbagliata” un’ipotesi (es. un letterale di troppo), limita la probabilità che l’errore resti alto dopo m campioni (union bound + $1 - x \leq e^{-x}$)
2. **Se vuoi dimostrare che NON è efficientemente learnable:** costruisci una riduzione da un problema NP-hard noto (es. 3-colorabilità) verso l’esistenza di un algoritmo di apprendimento efficiente – se l’apprendimento fosse efficiente, risolveresti il problema NP-hard in tempo polinomiale randomizzato

9 Concetti trasversali da ricordare sempre

- $P \subseteq NP \subseteq EXP$ (sempre vero, per ogni linguaggio)
- Se L è NP-completo e $L \in P \Rightarrow P = NP$ (creduto falso, ma non dimostrato)
- Ogni problema in P si riduce banalmente a qualsiasi problema NP-hard (riduzione “fasulla”, costruendo un’istanza fissa in base alla risposta già nota)
- NP-hard **non** implica essere in NP (es. HALT: NP-hard ma indecidibile)
- Un singolo esempio in due classi non dimostra che le classi siano uguali

10 Problemi NP-completi noti (il “kit di partenza” per le riduzioni)

Questi sono i problemi che abbiamo già dimostrato NP-completi durante il corso: possiamo usarli liberamente come punto di partenza per nuove riduzioni, senza doverli ridimostrare.

- **3SAT:** data una formula in 3-CNF, è soddisfacibile? (Il “capostipite”, dato per assodato via Teorema di Cook-Levin)

$$3SAT = \{F \mid F \in 3CNF \text{ e } F \text{ è soddisfacibile}\}$$

- **HAM-CYCLE:** dato un grafo, esiste un ciclo che passa per ogni nodo esattamente una volta?

$$HAMCYCLE = \{(V, E) \mid \exists \text{ un ciclo che tocca ogni } v \in V \text{ esattamente una volta}\}$$

- **HAM-PATH**: come sopra ma con un cammino (non necessariamente chiuso)

$$HAMPATH = \{(V, E) \mid \exists \text{ un cammino che tocca ogni } v \in V \text{ esattamente una volta}\}$$

- **TSP** (commesso viaggiatore): dato un grafo pesato e k , esiste un ciclo hamiltoniano di peso $\leq k$?

$$TSP = \{(G, t, k) \mid \exists \text{ ciclo hamiltoniano in } (G, t) \text{ di peso totale } \leq k\}$$

- **INDSET** (Independent Set): dato un grafo e k , esiste un insieme di k nodi a due a due non collegati?

$$INDSET = \{(G, k) \mid \exists I \subseteq V, |I| \geq k, \forall u, v \in I, (u, v) \notin E\}$$

- **VERTEX COVER**: dato un grafo e k , esiste un insieme di $\leq k$ nodi che copre ogni arco?

$$VERTEXCOVER = \{(G, k) \mid \exists W \subseteq V, |W| \leq k, \forall (u, v) \in E, u \in W \text{ o } v \in W\}$$

(riduzione da INDSET, via il lemma W è vertex cover $\iff V \setminus W$ è insieme indipendente)

- **CLIQUE**: dato un grafo e k , esiste un sottoinsieme di k nodi tutti collegati a due a due?

$$CLIQUE = \{(G, k) \mid \exists W \subseteq V, |W| \geq k, \forall u, v \in W, (u, v) \in E\}$$

(attenzione: 3CLIQUE con $k = 3$ fissato è invece in P!)

- **KNAPSACK**: dati pesi/valori di n oggetti e soglie z, k , esiste un sottoinsieme con peso $< z$ e valore $> k$?

$$KNAPSACK = \{(P, W, z, k) \mid \exists I, \sum_{i \in I} w_i < z \text{ e } \sum_{i \in I} p_i > k\}$$

(attenzione: con pesi tutti = 1, la variante VARKNAPSACK è in P!)

Come scegliere il problema di partenza giusto. Cerca un problema noto che “assomigli” concettualmente al tuo (stessa struttura combinatoria: sottoinsiemi, cammini, coperture...). Spesso basta una piccola variazione della struttura del problema noto per costruire la riduzione.